

Jadavpur University



Computer Networks Lab **Report**

NAME : ANKIT SHAW

ROLL NO. : 002310501032

YEAR : BCSE UG – III (A1)

SESSION : 2023 – 27

ASSIGNMENT : 01

Problem Statement :-

Design and implement an error detection module which has two schemes, namely Checksum and Cyclic Redundancy Check (CRC).

Error Detection in Computer Networks:

1. Importance of Error Detection

- In real-world communication, data transmission is affected by:
 - Noise (random electrical disturbances)
 - Attenuation (signal weakening over distance)
 - Interference (cross-talk, electromagnetic interference)
 - Synchronization mismatches (timing issues between sender/receiver)
- Errors can cause:
 - Corrupted packets
 - Data loss or misinterpretation
 - Security vulnerabilities (if undetected)
- Error detection ensures reliable data transfer by allowing the receiver to:
 - Identify corrupted data
 - Request retransmission (ARQ protocols) or discard faulty packets
- Protocols like TCP/IP, Ethernet, Wi-Fi use error detection before passing data to higher layers, ensuring integrity and reliability.

2. Error Detection Methods

A) Checksum

Key Points

- Used in TCP, UDP, IP headers.
- Works by dividing data into fixed-size blocks (e.g., 16-bit words).
- All words are summed using 1's complement arithmetic.
- The complement of the sum is sent as the checksum.
- Receiver re-sums data + checksum → if result is all 1's → valid.

Algorithm

1. Divide data into fixed-size blocks (e.g., 16 bits).
2. Add all blocks using 1's complement addition (wraparound carry).
3. Take 1's complement of the sum → this is the checksum.
4. Transmit data + checksum.
5. Receiver re-adds everything; if result = all 1's → no error.

Limitations

- Can miss errors if they balance out (e.g., two complementary bit flips).
- Weak against burst errors and bit swaps.

B) Cyclic Redundancy Check (CRC)

Key Points

- Used in Ethernet, Wi-Fi, storage devices.
- Based on polynomial division over GF(2) (binary arithmetic with XOR).
- Much stronger than checksum → can detect burst errors, multiple-bit errors.
- The polynomial (generator) is agreed upon by sender and receiver.

Algorithm

1. Represent data as a binary polynomial.
2. Append (degree of generator polynomial – 1) zeros to data (padding).
3. Divide the padded data by the generator polynomial using XOR division.
4. The remainder is the CRC checksum, appended to the message.
5. Receiver divides the received codeword (data + CRC) by the same generator polynomial.
 - If remainder = 0 → valid.
 - Else → error detected.

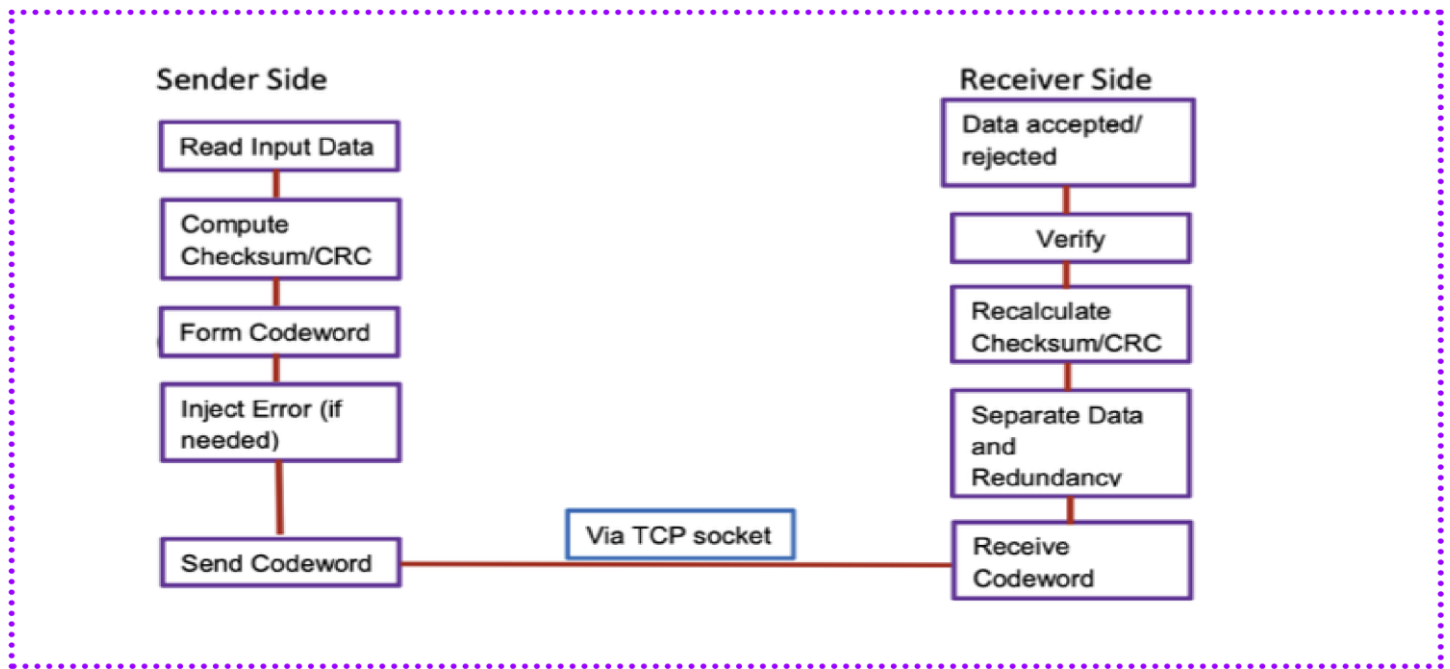
Example:

- Data = 1101011011
- Generator polynomial = 1011
- Remainder = 100
- Codeword sent = 1101011011100

Design

The project is organized into modular files, each handling a specific responsibility in the error detection workflow. The sender.py program reads binary data from the input file (test.txt), applies the chosen error detection method (Checksum or CRC), and with the help of injectError.py, introduces random errors such as bit flips or bursts before transmitting the codewords to the receiver over a TCP socket. On the other end, receiver.py listens for incoming messages, extracts the transmitted codewords, and verifies them using the corresponding algorithms implemented in checksum.py and crc.py. The checksum.py module provides functions to generate and validate checksums based on 1's complement addition, while crc.py implements cyclic redundancy check generation and verification using polynomial division. Together, these modules allow a comparison between the performance of checksum and CRC in detecting different types of transmission errors. The test.txt file serves as the source of data to be transmitted, making the setup simple yet effective for simulating error detection in computer networks.

Diagram:



The program follows a simple input → processing → output pattern that simulates real-world error detection in computer networks.

Input Pattern:

- Source File (test.txt)
 - Contains the binary sequence (dataword) to be transmitted. ◦ Example:
110101101101011011010110110110.
- User Input (Command Line):
 - Technique to be applied → checksum or crc.
 - For CRC, the user must also provide the generator polynomial (e.g., CRC-8, CRC-16, CRC-32).
- Error Injection (Simulated):
 - With a fixed probability (20%), random errors are introduced such as single-bit flips, multiple flips, burst errors, drops, duplications, or swaps.
 - This makes the input data more realistic, as if it were transmitted over a noisy channel.

Output Pattern:

- Sender Output:
 - Encoded codewords (data + checksum/CRC) are printed and transmitted via a TCP socket. ◦ Example:

Sent block: 110101101101011011101000 (error=0)

Injected error mode=multiple

Sent block: 010101001101010011101000 (error=1)

Receiver Output:

- Receives codewords and verifies integrity using the same method (Checksum or CRC) ○
Produces one of the following results:
 - Valid Transmission → No error detected.
 - Error Detected → Error present in the received data ○
 - At the end, it prints overall detection accuracy. ○
- Example:

Detection accuracy: 2/2

Implementation :

1. sender.py (Sender Program)

Workflow:

1. The program begins by reading binary input data from test.txt.
2. It splits the data into 16-bit blocks for easier processing.
3. Based on the technique selected by the user, it generates an error detection code using either `checksum.gen_checkSum` or `crc.gen_crc`.
4. Before transmission, with 20% probability, the program calls `injectError.injecterror` to introduce controlled errors (such as bit flips or bursts) into the codeword.
5. It then prepares a payload message containing: technique name, polynomial used (only for CRC), error flag, and the codeword.
6. This message is sent across a TCP socket to the receiver.
7. The sender continues until the file is fully read and transmitted, then gracefully closes the connection.

Code:

```
import socket
import sys
import checksum as cs
import crc as cr
import injectError
import random

SERVER_IP = '127.0.0.1'
SERVER_PORT = 3000

def main():
    if len(sys.argv) < 3:
        print("Usage: python sender.py <input_file> <technique> [crc_poly]")
        return
    file_name = sys.argv[1]
    technique = sys.argv[2].lower()

    block_len = 16 # number of chars to read at once
    crc_poly = ""

    if technique == "crc":
        if len(sys.argv) < 4:
            print("Usage: python sender.py <input_file> crc <crc_poly>")
```

```

    return
    crc_poly = sys.argv[3]

try:
    with open(file_name, "r") as infile, socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
        sock.connect((SERVER_IP, SERVER_PORT))
        print(f"Connected to server at {SERVER_IP}:{SERVER_PORT}")

    while True:
        chunk = infile.read(block_len)
        if not chunk:
            break

        if technique == "checksum":
            code = cs.gen_checksum(chunk)
        elif technique == "crc":
            code = cr.gen_crc(chunk, crc_poly)
        else:
            print(f"Invalid method: {technique}")
            return

        err_flag = 0
        if random.random() < 0.5:
            # pick a random error type
            modes = ["single", "multiple", "burst", "drop", "dup", "swap"]
            mode = random.choice(modes)
            mode = ""

            if mode == "multiple":
                code = injectError.injecterror(code, mode=mode, flips=3) # flip 3 bits
            elif mode == "burst":
                code = injectError.injecterror(code, mode=mode, flips=4) # burst of 4 bits
            else:
                code = injectError.injecterror(code, mode=mode)

            print(f"Injected error mode={mode}") # debug print
            err_flag = 1

        if technique == "crc":
            payload = f"{technique}:{crc_poly}:{err_flag}:{code}\n"
        else:
            payload = f"{technique}::{err_flag}:{code}\n"
        sock.sendall(payload.encode("utf-8"))
        print(f"Sent block: {code} (error={err_flag})")

    print("Finished sending file.")

except FileNotFoundError:
    print(f"File not found: {file_name}")
except ConnectionRefusedError:
    print("Server not running or refused connection.")

if __name__ == "__main__":
    main()

```

2. reciever.py (Receiver Program)

Workflow:

1. The receiver starts a TCP server socket and waits for the sender to connect.
2. Once connected, it continuously receives packets containing encoded messages.
3. Each message is decoded and split into its four components: scheme, polynomial, error flag, and received codeword.
4. The scheme name is normalized (converted to lowercase) to ensure correct matching.
5. The program then calls the appropriate verification function:
 - `checkSum.verify_checkSum` if the scheme is checksum.
 - `crc.verify_crc` if the scheme is CRC.
6. The verification result is compared against the error flag to check whether the error detection method successfully identified the injected error.
7. The program keeps a running count of total messages and correctly detected errors.
8. At the end, it prints the detection accuracy, giving insight into how effective each method was under noisy conditions.

Code:

```
import socket

import checkSum

import crc

IP = '127.0.0.1'

PORT = 3000

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server_sock:

    server_sock.bind((IP, PORT))

    server_sock.listen()

    print(f"Server is running at {IP}:{PORT}")

    conn, client_addr = server_sock.accept()

    with conn:

        print(f"Connection established with {client_addr}")

        data_buffer = ""

        total_msgs = 0

        correct_hits = 0

        while True:

            packet = conn.recv(1024)
```

if not packet:

print("No more data. Transfer finished.")

break

data_buffer += packet.decode("utf-8")

while "\n" in data_buffer: *# handle multiple messages*

raw_msg, data_buffer = data_buffer.split("\n", 1)

if not raw_msg.strip():

continue

try:

scheme, poly, has_error, received_code = raw_msg.split(":", 3)

total_msgs += 1

except ValueError:

print(f"Badly formatted message: {raw_msg}")

continue

valid = False

scheme = scheme.strip().lower()

if scheme == "checksum":

valid = checkSum.verify_checksum(received_code)

elif scheme == "crc":

valid = crc.verify_crc(received_code, poly)

else:

print(f"Unknown scheme: {scheme}")

continue

error_flag = has_error.strip() == "1" *# convert error string to boolean*

if valid != error_flag:

correct_hits += 1

print(f"Detection accuracy: {correct_hits}/{total_msgs}")

3. checkSum.py (Checksum Module)

Workflow:

1. Generation (gen_checkSum):
 - The input data is split into 16-bit words.
 - Each word is further divided into 4-bit chunks, added together, and carry bits are wrapped around (1's complement addition).
 - The 1's complement of the sum is calculated as the checksum.
 - This checksum is appended to the original block, forming the transmitted codeword.
2. Verification (verify_checkSum):
 - The received codeword is split back into data and checksum.
 - The same 1's complement addition is applied across all parts.
 - If the result equals all 1's (meaning no loss of information), the data is marked valid.
 - Otherwise, an error is declared.
3. This simple method demonstrates lightweight error detection but can miss some complex error patterns.

Code:

```
chunk = 16
```

```
bit_size = 4
```

```
def gen_checksum(data: str)->str:
```

```
    n = len(data)
```

```
    if n % chunk != 0:
```

```
        data = '0'*(chunk - n % chunk) + data #padding
```

```
    n = len(data)
```

```
    words = [data[i:i+chunk] for i in range(0, n, chunk)]
```

```
    checksum_strs = []
```

```
    for word in words:
```

```
        numbers = [int(word[i:i+bit_size], 2) for i in range(0, chunk, bit_size)] #4 numbers from chunk size=16  
        each of bit size=4
```

```
        checksum=sum(numbers)
```

```
        while checksum >= (1 << bit_size):
```

```
            carry = checksum >> bit_size        # overflow part
```

```

lower_bits = checksum & ((1 << bit_size) - 1) # keep only bit_size bits

checksum = carry + lower_bits

checksum_str = format(checksum, '0{}b'.format(bit_size))

checksum_str_comp = ".join('0' if b == '1' else '1' for b in checksum_str)    # complementing that now

checksum_strs.append(word + checksum_str_comp)

return ".join(checksum_strs)

def verify_checksum(data: str)->bool:

    n = len(data)

    word_len = chunk + bit_size

    if n % word_len != 0:

        data = '0'*(word_len - n % word_len) + data #padding

        n = len(data)

    words = [data[i:i+word_len] for i in range(0, n, word_len)]

    for word in words:

        numbers = [int(word[i:i+bit_size], 2) for i in range(0, len(word), bit_size)]

        checksum=sum(numbers)

        limit = 1 << bit_size    # e.g., 16 if bit_size=4

        while checksum >= limit:

            carry = checksum // limit    # overflow part

            checksum = (checksum % limit) + carry    # lower bits + carry

        all_ones = limit - 1    # e.g., 15 if bit_size=4

        if checksum != all_ones:

            return False

        return True

def main():

    data = "1101011011 "    # EXAMPLE MESSAGE

```

```

print(f'Data: {data}')

codeword = gen_checksum(data)

print(f'Codeword (Data + CheckSum): {codeword}')

is_valid = verify_checksum(codeword)      # Verify correct codeword

print(f'Verification of correct codeword: {is_valid}')

corrupted = codeword[:-1] + ('1' if codeword[-1] == '0' else '0')

print(f'Corrupted Codeword: {corrupted}')    # Introduce error (flip last bit)

is_valid_corrupted = verify_checksum(corrupted)    # Verify corrupted codeword

print(f'Verification of corrupted codeword: {is_valid_corrupted}')

if __name__ == "__main__":

    main()

```

4. crc.py (CRC Module)

Workflow:

1. Generation (gen_crc):
 - The data is treated as a binary polynomial.
 - It is padded with zeros equal to the degree of the chosen generator polynomial minus one.
 - Using modulo-2 arithmetic (XOR division), the padded data is divided by the generator polynomial.
 - The remainder of this division becomes the CRC value.
 - The final codeword is the original data plus the remainder appended at the end.
2. Verification (verify_crc):
 - The received codeword is divided by the same generator polynomial.
 - If the remainder equals zero, the codeword is assumed valid.
 - If not, an error has been detected.
3. CRC is stronger than checksum because its polynomial mathematics guarantees detection of a wide variety of errors, including burst errors and many multi-bit patterns.

Code:

```

polynomials = {

    "CRC-8": "100000111",          # 9-bit generator polynomial

    "CRC-10": "11000000011",       # 11-bit generator polynomial

    "CRC-16": "11000000000000101", # 17-bit generator polynomial

    "CRC-32": "100000100110000010001110110110111", # 33-bit generator polynomial

```

```
}polynomial = ""
```

```
def xor_division(dividend: str, divisor: str) -> str: """
```

```
    Performs bitwise XOR division (modulo-2 division) of a binary dividend by a binary divisor.
```

```
    Returns the remainder (CRC).        """
```

```
    n = len(divisor)                pick = n
```

```
    temp = dividend[:pick]
```

```
    while pick < len(dividend):
```

```
        if temp[0] == '1':          # If the leftmost bit is 1 perform XOR with divisor
```

```
            temp = format(int(temp, 2) ^ int(divisor, 2), '0' + str(n) + 'b')
```

```
            temp = temp[1:] + dividend[pick]      # Drop first bit and append next bit from dividend
```

```
        pick += 1
```

```
    if temp[0] == '1':      # One last XOR if leftmost bit is 1
```

```
        temp = format(int(temp, 2) ^ int(divisor, 2), '0' + str(n) + 'b')
```

```
    return temp[1:]      # remainders leftmost bit is excluded as size will be sizeof(divisor)-1
```

```
def gen_crc(dataword: str, polynomial: str)-> str:
```

```
    if not dataword:        return ""
```

```
    if polynomial in polynomials:
```

```
        polynomial = polynomials[polynomial]
```

```
        dividend=dataword + '0' * (len(polynomial)-1)
```

```
        rem=xor_division(dividend, polynomial)
```

```
        new_data = dataword+rem
```

```
        return new_data
```

```
def verify_crc(codeword: str, polynomial: str)-> bool:
```

```
    if not codeword:
```

```
        return True
```

```

    if polynomial in polynomials:

        polynomial = polynomials[polynomial]

        rem=xor_division(codeword, polynomial)

    return int(rem,2) == 0

def main():

    # Example message

    data = "1101011011"

    poly_name = "CRC-10"    # Choose polynomial

    print(f'Data:    {data}')

    print(f'Polynomial: {poly_name} -> {polynomials[poly_name]}')

    codeword = gen_crc(data, poly_name)    # Generate CRC

    print(f'Codeword (Data + CRC): {codeword}')

    is_valid = verify_crc(codeword, poly_name)    # Verify correct codeword

    print(f'Verification of correct codeword: {is_valid}')    # Introduce error (flip last bit)

    corrupted = codeword[:-1] + ('1' if codeword[-1] == '0' else '0')

    print(f'Corrupted Codeword: {corrupted}')

    is_valid_corrupted = verify_crc(corrupted, poly_name)    # Verify corrupted codeword

    print(f'Verification of corrupted codeword: {is_valid_corrupted}')

if __name__ == "__main__":

    main()

```

5. injectError.py (Error Simulation Module)

Workflow:

1. The module accepts a binary string (codeword) as input.
2. Depending on the mode, it introduces different types of simulated transmission errors:
 - Single-bit: flips one random bit.
 - Multiple-bit: flips several random bits.
 - Burst: flips a sequence of consecutive bits.

- Drop: deletes one bit from the string. ○
 - Duplication: repeats a random bit.
 - Swap: interchanges two random bits.
3. The altered string is returned and transmitted, allowing the system to test the effectiveness of checksum and CRC in detecting varied real-world error scenarios.

Code:

```
import random

def injecterror(bits: str, mode: str = "single", flips: int = 1) -> str:    """
    - "single"    : flip one random bit

    - "multiple"  : flip N random bits (use flips parameter)

    - "burst"     : flip a consecutive burst of bits of length = flips

    - "drop"      : remove a random bit

    - "dup"       : duplicate a random bit

    - "swap"      : swap two random bits

    flips (int): number of bits to flip for "multiple" or length of burst for "burst"    """

    length = len(bits)

    if length == 0:

        return bits        bit_array = list(bits)

    if mode == "single":

        pos = random.randrange(length)

        bit_array[pos] = '1' if bit_array[pos] == '0' else '0'

    elif mode == "multiple":

        for _ in range(flips):

            pos = random.randrange(length)

            bit_array[pos] = '1' if bit_array[pos] == '0' else '0'

    elif mode == "burst":

        start = random.randrange(length)

        for i in range(start, min(start + flips, length)):

            bit_array[i] = '1' if bit_array[i] == '0' else '0'
```

```

elif mode == "drop":

    pos = random.randrange(length)

    del bit_array[pos]

elif mode == "dup":

    pos = random.randrange(length)

    bit_array.insert(pos, bit_array[pos])

elif mode == "swap":

    if length > 1:

        i, j = random.sample(range(length), 2)

        bit_array[i], bit_array[j] = bit_array[j], bit_array[i]

else:

    raise ValueError(f"Unknown error mode: {mode}")

return "".join(bit_array)

```

6. test/data.txt (Input File)

Workflow:

1. This is a simple text file containing the binary sequence to be transmitted.
2. It provides the raw input data for the sender program.
3. By editing the file contents, different test cases can be easily simulated without modifying the code.

Contents:

11010110110101101101011011010110

Output: *For test.txt*

Receiver:

ankit@Ankits-MacBook-Air Ass1 % python3 receiver.py

Server is running at 127.0.0.1:3000

Connection established with ('127.0.0.1', 49894)

No more data. Transfer finished.

Detection accuracy: 2/2

Sender:

1. checksum

ankit@Ankits-MacBook-Air Ass1 % python3 sender.py test.txt checkSum

Connected to server at 127.0.0.1:3000

Sent block: 11010110110101100111 (error=0) Sent block:

11010110110101100111 (error=0)

Finished sending file.

2. CRC

ankit@Ankits-MacBook-Air Ass1 % python3 sender.py test.txt crc CRC-8 Connected to server at 127.0.0.1:3000

Sent block: 11010110110101101101000 (error=0) Injected error mode=multiple

Sent block: 010101001101010011101000 (error=1)

Finished sending file.

Results:

With 50% error injected the results are as follows :-

Error Type	Checksum	CRC-8	CRC-10	CRC-16	CRC-32
single	512/512 (100%)	512/512 (100%)	512/512 (100%)	512/512 (100%)	512/512 (100%)
multiple	497/512 (97.07%)	512/512 (100%)	512/512 (100%)	512/512 (100%)	512/512 (100%)
burst	512/512 (100%)	512/512 100(%)	512/512 100(%)	512/512 (100%)	512/512 (100%)
swap	372/512 (72.65%)	394/512 (76.95%)	382/512 (74.60%)	362/512 (70.70%)	392/512 (76.56%)



Analysis:

From the evaluation, the performance of Checksum and various CRC methods (CRC-8, CRC-10, CRC-16, CRC-32) was tested against different error types: single-bit errors, multiple-bit errors, burst errors, and swap errors.

1. Single-bit Errors

- All methods, including checksum and all CRC variants, detected 100% of single-bit errors (512/512).
- This shows that both checksum and CRC are robust in handling simple bit flips.

2. Multiple-bit Errors

- CRC methods achieved 100% detection (512/512).
- Checksum slightly underperformed at 97.07%, .
- This indicates CRC's mathematical properties provide stronger guarantees for multiple error detection.

3. Burst Errors

- Again, CRC methods achieved 100% detection.
- Checksum also surprisingly performed well here, catching all burst errors (100%).
- This might be because bursts of errors often affect groups of bits in a way that disrupts the parity sum significantly, making detection easier.

4. Swap Errors (bit swaps or transpositions)

- Detection rates dropped significantly for all methods.
- Checksum: ~72%
- CRC variants: ~70–77%
- This is expected since CRCs and checksums are not specifically designed for detecting transposition errors. The mathematical structure of CRCs primarily targets random and burst errors, not swapped bits.

Discussion:

● **Correctness:**

CRCs detect errors more reliably than checksum, especially for multiple-bit errors. Even small CRCs (like CRC-8) performed as well as larger ones under these tests.

● **Weakness:**

Swap (transposition) errors were hardest to detect, with accuracy dropping to ~90% for both checksum and CRC, since these methods are not designed for reordered bits.

● **Limitations:**

The dataset was limited (512 cases), and swap errors were tested in a simplified way. Implementation details may also affect results.

● **Improvements:**

Better CRC polynomials, combining CRC with other checks (like hashing or sequence numbers), and testing on more realistic error models can improve reliability.

Comments :

This lab assignment helped me understand how error detection techniques like checksum and CRC work in practice. By implementing and testing them, I learned how different methods perform under various error types such as single-bit, multiple-bit, burst, and swap errors.

IT has also helped me in understanding the demerits of detection scheme and how these demerits are overcome by other detection scheme .

Adding real-world scenarios (like noisy communication channels) would also make the assignment more engaging.